UNIVERSIDAD PERUANA CAYETANO HEREDIA

FACULTAD DE CIENCIAS E INGENIERÍA
CARRERA DE INGENIERÍA INFORMÁTICA



PROYECTO PARCIAL

Programación Orientada a Objetos

Fuga de la Prisión

Diseño e implementación de un videojuego 2D

Curso:	Programación Orientada a Objetos
Docentes:	MSc. Wilder Nina Choquehuayta (principal) MSc. Percy Lovón Ramos (auxiliar) Dr. Ortiz Ortiz Emanuel (auxiliar)
Tema asignado:	Fuga de la Prisión
Ciclo:	IV
Integrantes:	JOSE FERNANDO CHAVEZ LOZANO MICKELE CARLOS TORRES VELAZCO PIERO CEMEI ROMERO MEZA RICHARD AARON SORIANO CORDOVA
Período académico:	2026-II

Lima — Perú

2026

Informe parcial: Fuga de la Prisión

I. Resumen Ejecutivo

Fuga de la Prisión es un videojuego 2D de escape y sigilo con cuatro niveles y un jefe. La arquitectura separa personajes, mapa, puertas, ruido, estados, puntaje, combate y dibujo. Sus 25 clases aparecen en el UML y aplican asociación, agregación, composición, encapsulamiento, herencia y polimorfismo. State organiza los estados y Observer comunica el puntaje. El avance incluye una celda inicial, alarmas que alertan a los guardias y un jefe con tres capas de vida. Las pruebas y capturas comprueban estas funciones.

II. Introducción

a. Contexto y objetivo

El proyecto evalúa la aplicación de la Programación Orientada a Objetos mediante un videojuego. La temática asignada es Fuga de la Prisión: escapar evitando guardias, cámaras y sistemas de seguridad [1]. La entrega parcial busca demostrar un diseño de clases coherente, sus fundamentos POO y los patrones implementados, acompañados de una interfaz y funciones que se puedan ejecutar [2].

La campaña contiene tres zonas de sigilo y una arena contra el Alcaide. En el primer nivel se empieza encerrado y se fuerza la puerta manteniendo E. Cada zona exige conseguir una llave y alcanzar la salida. Los pasos pueden atraer patrullas; las cámaras alertan a todos los guardias y la captura ocurre por contacto. El último mapa de sigilo tiene galerías horizontales y atajos que se abren con su llave.

b. Tecnologías utilizadas

Tecnología	Uso
Python y Pygame	Reglas, ventana, teclado y dibujo de los personajes stickman. Verificado con Python 3.13.7 y Pygame 2.6.1.
Mermaid.js	Fuente editable del UML e imagen generada desde el mismo modelo.
unittest	Pruebas de juego, celda, alarmas, paredes, campaña, tres capas del jefe y correspondencia entre código y documentos.
Git y GitHub	Registro y publicación de versiones. Se revisa el historial real del repositorio.

III. Diseño de Arquitectura Conceptual y de Clases

a. Diagrama de clases UML

El modelo incluye las 25 clases del juego, conforme a la indicación del docente de representar todas. Cada caja tiene nombre, atributos propios y operaciones. Los atributos que guardan objetos señalan su clase: mapa: Mapa, celda: Optional<PuertaCelda>, ruidos: SistemaRuido, carrera: HabilidadCarrera y jefe: Alcaide. Los miembros heredados se leen en la base. Las propiedades se muestran como atributos; los constructores se incluyen en la guía completa y se omiten en el UML para reducir su tamaño.

El diagrama completo está en la última página ampliada. modelo_parcial.mmd es la fuente editable y modelo_parcial.png se genera con Mermaid.js. Las clases de pruebas no son componentes del videojuego.

b. Estructura de componentes

Archivo y clases	Responsabilidad
juego.py: Juego	Coordina la campaña, conserva puntos y delega en el estado actual.
estados.py: EstadoJuego, Jugando, Pausa, Derrota, Victoria, NivelCompletado	Definen entrada, actualización, pausa y transición entre niveles.
nivel.py: Nivel, PuertaCelda, Camara, Llave, PuertaSalida	Coordinan la celda, alertas, captura, recogida de llaves, atajos, premios y salida.
mapa.py: Mapa	Conserva el plano, abre puertas, resuelve colisiones y busca caminos.
personajes.py: Personaje, Prisionero, Guardia, HabilidadCarrera	Movimiento común, controles, patrulla, investigación, persecución y carrera.
ruido.py: SistemaRuido	Registra a los guardias y les notifica los pasos que llegan por los pasillos.
puntuacion.py: Puntaje, ObservadorPuntaje, PanelPuntaje	Validan los puntos y comunican sus cambios mediante Observer.
combate.py: Proyectil, Alcaide, Combate	Disparos, tres capas de vida, fases, avisos, impactos e inmunidad.
vista.py: Vista	Dibuja mapas, celda, cámaras ancladas, ruido, alertas y barras del jefe.

Lectura de las relaciones

El rombo negro representa composición y queda junto al objeto que crea sus partes; el blanco representa agregación. El triángulo vacío apunta a la clase base. La flecha continua indica una referencia guardada y la discontinua un uso o dependencia. 1, 0..1 y 0..* significan uno, ninguno o uno, y ninguno o varios. + indica acceso público; # señala uso interno por convención de Python.

IV. Aplicación de Fundamentos de POO

a. Asociación, agregación y composición

Asociación. Personaje recibe y guarda su Nivel para consultar datos y colisiones. SistemaRuido conserva el Mapa recibido, sin crear otro:

```
self.nivel = nivel
self.mapa = mapa
```

La primera instrucción pertenece a Personaje y la segunda a SistemaRuido.

Agregación. Juego crea Puntaje y lo entrega al Nivel. El mismo objeto se conserva al cambiar de nivel. SistemaRuido también guarda guardias que Nivel creó y registró como oyentes:

```
self.nivel = Nivel(self.puntaje, self.indice_nivel)
self.ruidos.registrar(guardia)
```

Composición. Nivel crea su Mapa, SistemaRuido, personajes, cámaras, llave y salida. Solo en el primer nivel crea PuertaCelda; en la arena crea Combate. Prisionero crea HabilidadCarrera y Combate crea Alcaide y Proyectil:

```
self.mapa = Mapa(PLANOS[indice])
self.celda = PuertaCelda(centro(4, 11)) if indice == 0 else None
```

b. Herencia y polimorfismo

Prisionero y Guardia heredan de Personaje. Comparten posición y movimiento, pero cada uno implementa update: el prisionero responde al teclado y emite pasos; el guardia percibe, decide y sigue caminos. Nivel utiliza una llamada común:

```
for personaje in self.personajes:
    personaje.update()
```

Juego delega actualizar y manejar_entrada en su EstadoJuego activo. Los cinco estados concretos responden según la situación. PanelPuntaje hereda de ObservadorPuntaje e implementa actualizar_puntaje.

c. Encapsulamiento y abstracción

Puntaje conserva _valor y lo modifica mediante set_valor, sumar y reiniciar. El setter valida el dato y notifica el cambio. Personaje consulta sus coordenadas mediante get_posicion y las modifica con mover, que respeta las colisiones. El guion bajo expresa uso interno; no impide por sí solo el acceso en Python.

```
def get_valor(self):
    return self._valor

def set_valor(self, valor):
    if type(valor) is not int or valor < 0:
        raise ValueError("El puntaje debe ser un entero no negativo.")
    self._valor = valor
    self._notificar()
```

Personaje, EstadoJuego y ObservadorPuntaje son abstractas: sus clases hijas deben implementar las operaciones indicadas. Esto permite compartir una base y conservar el comportamiento propio de cada objeto.

Patrones de diseño implementados

State. EstadoJuego es la base de Jugando, Pausa, Derrota, Victoria y NivelCompletado. Juego delega en el estado activo. La pausa detiene movimiento, celda, vigilancia, temporizadores y combate.

Observer. Puntaje notifica a PanelPuntaje al cambiar su valor. SistemaRuido utiliza una comunicación semejante: registra oyentes y llama escuchar cuando les llega un paso; el guardia decide su conducta.

VI. Implementación de Resultados Parciales

Se juega con WASD o flechas, Ctrl reduce el ruido y Shift permite correr. Al inicio se debe mantener E durante dos segundos junto a los barrotes para salir de la celda. Las cámaras están alineadas con las paredes: su alarma alerta a todos los guardias y no causa derrota instantánea. Los guardias investigan, persiguen y buscan en la última posición conocida; solo capturan por contacto.

El Patio tiene tres galerías horizontales, tres patrullas y tres cámaras. La llave del ala norte abre dos atajos de regreso. Las puertas cambian tanto el dibujo como las colisiones y los caminos. En la arena, el Alcaide dispone de tres capas de 72 puntos: la llave aparece únicamente tras agotar las tres.

Fase	Coronas	Velocidad máxima de la corona	Espera tras atacar
1: tres capas restantes	12 proyectiles	115 píxeles/s	1,45 s
2: dos capas restantes	40 proyectiles	155 píxeles/s	0,85 s
3: una capa restante	84 proyectiles	205 píxeles/s	0,45 s

Las ráfagas y embestidas también aceleran. Cada cambio de capa se anuncia durante 1,1 segundos; se retiran los proyectiles y se evita perder varias capas en el mismo instante.





Figura 1. Celda inicial cerrada. Figura 2. Última capa del jefe, con la fase 3 activa. Capturas del juego ejecutado durante la verificación.

Resultados comprobados y uso de Git

El 15 de octubre de 2026 se ejecutaron 54 pruebas automáticas sin fallos. Cubren celda, apertura de atajos, alarmas, paredes, conectividad, ruido, persecución, captura, puntaje, estados y combate con tres capas. También comparan las clases, atributos, métodos y código de la guía con el UML. La verificación completó los cuatro niveles mediante movimiento y disparos, sin desactivar enemigos, y generó catorce capturas.

La llave concede 100 puntos, cada salida 500 y el jefe 1000: la campaña termina con 3400. El jefe fue vencido en 72,52 segundos de simulación. R repite el nivel y restaura los puntos de entrada; N empieza desde cero. La rúbrica pide mostrar un avance del 25 % con interfaces y funciones [2], pero no da una fórmula para medirlo; aquí se presentan funciones comprobables.

Git. Hasta 8eed979, el historial contiene trece commits: once con el autor configurado JhosFcL, uno con Simei y uno con shzjeud. Registran planificación, implementación y documentación. Estos nombres no identifican por sí solos el trabajo de los cuatro integrantes. El detalle se conserva en docs/evidencias/git_estado.md y cada integrante debe explicar sus aportes reales. Los cambios de esta revisión quedan preparados para que el equipo los registre y publique.

VII. Conclusiones

El avance reúne cuatro niveles, apertura de celda, sigilo, ruido de pasos, investigación, persecución, llaves, atajos, puntaje y un jefe con tres capas de dificultad creciente. Sus 25 clases aparecen en el UML y están explicadas con su código. Se pueden demostrar los seis fundamentos POO y State y Observer.

La dificultad principal fue mantener coherentes puertas, paredes, sonido y caminos. Separar Mapa, PuertaCelda y SistemaRuido de la coordinación de Nivel permitió comprobar cada regla. Las pruebas de alarmas y cambios de capa evitaron derrotas y premios anticipados. El equipo debe ensayar la partida con teclado y preparar la explicación oral.

VIII. Anexos

Fragmento 1. La apertura de la celda modifica el mismo mapa que utiliza el movimiento:

```
if self.celda and self.celda.actualizar(
        dt, self.prisionero, "interactuar" in self.teclas):
    self.mapa.abrir_puerta(self.celda.posicion)
    self.ruidos.emitir(self.celda.posicion, 260)
```

Fragmento 2. La fase depende de las capas que conserva el jefe:

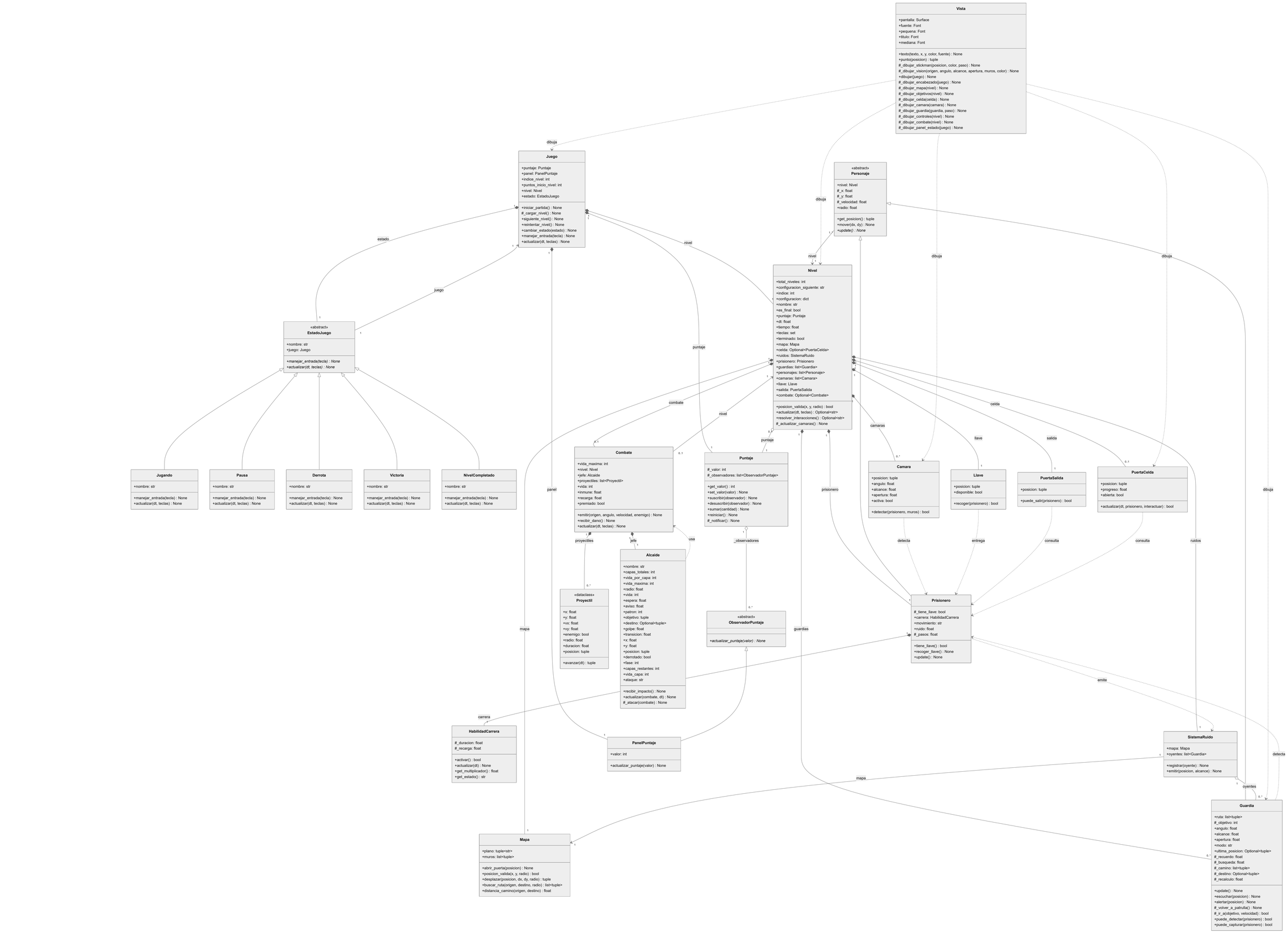
```
return min(self.capas_totales,
           self.capas_totales - self.capas_restantes + 1)
```

El UML completo está en la página ampliada siguiente. Explicacion_de_clases.md reúne las 25 clases y sus bloques completos. docs/evidencias conserva las capturas y la salida de las pruebas.

Material del curso

[1] Proyecto Final POO, ciclo 2026-2, temática 13.
[2] Rúbrica y Estructura - Proyecto Parcial, ciclo 2026-2.

VIII. Anexos - Diagrama UML completo: 25 clases del juego



Nombre, atributos propios y operaciones. Propiedades como atributos; miembros heredados en la base.
Rombo negro: composición | Rombo blanco: agregación | Triángulo: herencia | Flechas: asociación y dependencia